

VOLUME 9

COMPLETE VOLUME — CHAPTERS 9.1–9.9

Quality Assurance

Ensuring Reliability — From Error Taxonomy to the Device Matrix That Gates Every Release

Document Title	Volume 9 — Quality Assurance (Combined)
Volume Reference	Volume 9
Chapters Included	9.1 through 9.9 (9 chapters)
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; grounded in Volumes 0, 1, 3, 5, and 6

Table of Contents

This volume ensures reliability: an error taxonomy verifying Volume 6's error-handling mechanism is fully covered, logging and analytics (the latter grounded directly in Volume 0's own Vision Document success metrics), measurable performance targets, and the full testing pyramid from unit tests through the device matrix that gates every release.

Chapter	Title
9.1	Error Handling
9.2	Logging
9.3	Analytics
9.4	Performance Requirements
9.5	Testing Strategy
9.6	Unit Testing
9.7	Integration Testing
9.8	Manual Testing
9.9	Device Testing

VOLUME 9 — QUALITY ASSURANCE

CHAPTER 9.1

Error Handling

The QA Lens on Volume 6, Chapter 6.9's Mechanism — Coverage, Not Implementation

Document Title	Error Handling (QA)
Chapter Reference	9.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Companion To	Volume 6, Chapter 6.9 (implements the mechanism this chapter verifies)

1. Purpose — Not a Duplicate of Volume 6.9

Volume 6, Chapter 6.9 built the mechanism: sealed failure types, global uncaught-exception handlers, Crashlytics reporting. This chapter asks the QA question that mechanism alone can't answer: how do we know every failure mode is actually modeled, actually surfaces the right message, and never silently swallowed? That's a test-coverage and error-taxonomy question, which is this volume's job.

2. Error Taxonomy — What Must Be Covered

Class	Example	Required Test Coverage
Modeled/expected (sealed failure types)	ChecklistFailure.batteryTooLow (Volume 3, Ch.3.9)	Unit test per failure variant asserting the exact Chapter 2.9 copy renders — Chapter 9.6.
Network/backend errors	401 expired token, 5xx from a Lambda	Integration test forcing each status code via a fake server — Chapter 9.7.
Uncaught/unexpected	A plugin throwing an undocumented platform exception	Cannot be unit-tested by definition; covered by Crashlytics monitoring in production (Volume 6.9) and a manual exploratory pass (Chapter 9.8) attempting to break each screen.
Silent failure (the worst case)	An error that neither surfaces to the user nor reaches Crashlytics	Explicitly tested for: every repository method's error path has a unit test asserting it reaches an AsyncError, not a swallowed exception or a hung Future.

3. The One Rule This Chapter Enforces

No untested error path ships: any pull request adding a new failure mode (a new sealed subtype, a new possible API error code) must add its corresponding test in the same PR — enforced at code review (Volume 3, Chapter 3.7's checklist already lists this; this chapter is where the requirement is defined, not just referenced).

4. What This Chapter Defers

- Exact unit/integration test mechanics — Chapters 9.6/9.7.
- Crash reporting operational process — Volume 10, Chapter 10.6.

VOLUME 9 — QUALITY ASSURANCE

CHAPTER 9.2

Logging

What Gets Logged, at What Level, and for How Long

Document Title	Logging
Chapter Reference	9.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Extends	Volume 3, Chapter 3.7, Section 6 (Logging)

1. Log Levels

Level	Used For	Example
debug	Verbose local development only, stripped from release builds	Provider rebuild traces during active feature work.
info	Normal lifecycle events worth seeing in production logs	Session started, chunk finalized, upload complete.
warn	Recoverable but noteworthy conditions	Upload retry attempt N of 6 (Volume 5, Ch.5.13).
error	A failure that reached a user-visible or Crashlytics-reported state	Checklist failure, upload permanently Failed.

2. Correlation IDs

Every log line touching a chunk or session includes its `chunk_id/session_id` (Volume 4, Chapter 4.4's identifiers) — this is what makes it possible to reconstruct one Collector's one session's full journey (record → chunk → queue → upload → verify) across the mobile app's local logs and the backend's CloudWatch logs, without guessing which log lines belong together.

3. Where Logs Go

Mobile: structured logs (Volume 3, Chapter 3.7's logger) are kept in a local ring buffer (last N entries) attached automatically to a Crashlytics report if a crash occurs, and are not otherwise transmitted off-device — logging is for local debugging and crash context, not a telemetry pipeline in itself (that's Chapter 9.3, Analytics, a deliberately separate concern).

Backend: every Lambda's structured logs go to CloudWatch (Volume 4, Chapter 4.9), queryable by correlation ID via CloudWatch Insights.

4. Redaction

- GPS coordinates, device identifiers, and any field Volume 8, Chapter 8.6 classifies as personal data are never logged at info/debug level in a form that would leak them into a general-purpose log stream — logs reference a `chunk_id` and let a developer join against the metadata store (Volume 4) if genuinely needed, rather than duplicating sensitive fields into logs by default.
- This mirrors, and is enforced by the same review discipline as, Volume 6 Chapter 6.9's crash-report scrubbing rule.

5. Retention

Log retention (90 days) already matches Volume 8, Chapter 8.7, Section 1's table — restated here as the operational configuration (CloudWatch log group retention policy) that implements that policy decision, not a new number.

VOLUME 9 — QUALITY ASSURANCE

CHAPTER 9.3

Analytics

Measuring the Exact Success Metrics Volume 0's Vision Document Set

Document Title	Analytics
Chapter Reference	9.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Grounds	Volume 0, Chapter 0.1 (Vision Document), Section on Success Metrics

1. Purpose

Volume 0's Vision Document named four success metrics before any architecture existed. This chapter is where those metrics finally get a concrete data source — every one of them is computable directly from data this system already collects (Volume 4's chunks/sessions tables), not a new tracking system bolted on afterward.

2. Vision Document Metrics → Data Source

Vision Metric	Computed From
Upload success rate	<code>chunks.status = 'complete'</code> ÷ total chunks, per week (Volume 4, Chapter 4.4).
Data-loss incidents	Chunks that reach a terminal Failed state with no successful retry, cross-referenced against Chapter 5.13's retry-exhaustion event — tracked per Collector and app version.
Chunk upload latency (median/p95)	<code>chunks.created_at</code> → <code>chunk_metadata.verified_at</code> , aggregated in the backend (no new instrumentation needed beyond what Volume 4 already timestamps).
Recovery time after network loss	Time between a logged connectivity-restored event (Volume 5, Chapter 5.12's <code>ConnectivityService</code>) and the next chunk status transition out of Queued.

3. Event Tracking (Product Analytics)

- Firebase Analytics (already in the stack, Volume 4 Chapter 4.1) is used for lightweight product events distinct from the operational metrics in Section 2 — screen views, checklist pass/fail rate, retry-button tap rate — useful for UX iteration (Volume 2) but not for the reliability metrics above, which are derived from the authoritative backend data instead of client-reported events (a client-reported "upload succeeded" event could itself be wrong or lost; the backend's own `chunks.status` is the source of truth).
- No analytics event ever includes raw GPS coordinates or footage content — only IDs and categorical/numeric values, consistent with Volume 8, Chapter 8.6's data-minimization stance.

4. Reporting

A weekly dashboard (built from the Section 2 queries) is the operational read-out against Volume 0's own stated targets — this is the mechanism that lets the project owner actually answer "are we hitting the Vision Document's goals" with real numbers rather than impression.

5. What This Chapter Defers

- The dashboard's exact visual form — a Volume 10 (Monitoring) or later implementation detail, not fixed here.

VOLUME 9 — QUALITY ASSURANCE

CHAPTER 9.4

Performance Requirements

Measurable Targets, Not Just Volume 1's Prose

Document Title	Performance Requirements
Chapter Reference	9.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Expands	NFR-META-01, NFR-AVL-02 (Volume 1)

1. Targets

Area	Target	Measured By
Metadata generation overhead	< 500ms per chunk (NFR-META-01, restated exactly)	Instrumented timestamp diff around Volume 5, Chapter 5.7's generation step.
Upload resume after reconnect	< 30s (NFR-AVL-02, restated exactly)	ConnectivityService event → next status transition (same data source as Chapter 9.3's recovery-time metric).
App cold start to Login screen	< 2.5s on a mid-range device	Manual stopwatch during Chapter 9.9's device matrix testing; no automated instrumentation needed for a one-time-per-launch metric.
Recording Screen frame rate	Sustained 60fps, zero dropped-frame jank during the full 10-minute chunk	Flutter DevTools' performance overlay during Chapter 9.8's manual device testing.
Battery drain during continuous recording	< 15% per 10-minute chunk on a reference mid-range device	Manual measurement, Chapter 9.9.
Memory ceiling during Recording	No OOM kill across a full multi-chunk session on the reference low-end device	Android Studio / Xcode Instruments profiling during device testing.

2. Why These Specific Targets

Frame rate and battery matter more here than in a typical app: the Recording Screen (C-09) runs continuously for up to 10 minutes per chunk, often back-to-back across a full field shift — a janky preview or excessive battery drain isn't a minor annoyance here, it's a direct threat to the product's core promise of reliable, unattended field capture.

Cold-start time matters for adoption, not just polish: Collectors are field workers, not developers — a slow-feeling app on a mid-range device erodes trust in the tool faster than the same slowness would in an internal admin dashboard.

3. What This Chapter Defers

- The device matrix these targets are actually measured against — Chapter 9.9 (Device Testing).
- How performance regressions are caught before release — Chapter 9.5's testing strategy, and Volume 10's release process gate.

VOLUME 9 — QUALITY ASSURANCE

CHAPTER 9.5

Testing Strategy

The Pyramid Everything in Chapters 9.6–9.9 Fits Into

Document Title	Testing Strategy
Chapter Reference	9.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	The tooling named in Volume 3, Chapter 3.1

1. The Pyramid

Layer	Volume	Proportion of Suite	Tooling
Unit	9.6	~70%	flutter_test, mocktail (Volume 3, Ch.3.1)
Integration / Widget	9.7	~20%	integration_test, golden_toolkit
Manual / Exploratory	9.8	~8%	Human tester, real devices
Device Matrix	9.9	~2% (but gates every release)	Physical devices + Firebase Test Lab

More weight sits at the bottom of the pyramid deliberately — Volume 3, Chapter 3.4's layered architecture (Domain/Data as pure Dart) is what makes a 70% unit-test proportion realistic rather than aspirational; most business logic (every BR-tagged rule) is testable without a widget tree or a real device at all.

2. Coverage Targets

- Domain layer (use-cases): 90%+ line coverage — this is where BR-01–23 actually live in code, and Volume 3, Chapter 3.7's review checklist already requires a test for any new business rule.
- Data layer (repositories): 80%+, focused on error-path coverage per Chapter 9.1's rule, not just the happy path.
- Presentation layer (widgets): golden tests for every Design System component (Volume 2, Chapter 2.8) rather than a blanket coverage percentage — pixel-consistency matters more here than line coverage.

3. CI Gating

Volume 7, Chapter 7.13 already named this: every PR runs the full unit + integration suite; a release build (Volume 10) additionally requires the Chapter 9.9 device matrix to pass before store submission. No layer of this pyramid is optional — a missing test for a new business rule is a code-review blocker, not a suggestion.

4. What Volume 9's Remaining Chapters Cover

- 9.6 — unit testing specifics and examples.
- 9.7 — integration/end-to-end flows and golden tests.
- 9.8 — the manual/exploratory test plan for what automation can't reach (real camera hardware, real background upload).
- 9.9 — the device/OS matrix release testing runs against.

VOLUME 9 — QUALITY ASSURANCE

CHAPTER 9.6

Unit Testing

Testing Domain and Data, With No Widget Tree and No Real Device

Document Title	Unit Testing
Chapter Reference	9.6
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. What Gets a Unit Test

- Every use-case class (Volume 3, Chapter 3.4's Domain layer) — one test file per class, per Volume 3 Chapter 3.6's test-mirrors-lib rule.
- Every Repository's error paths (Chapter 9.1's rule) — not just its happy path.
- Every Riverpod Notifier's state transitions, using ProviderContainer overrides (Volume 6, Chapter 6.4) rather than a widget pump.

2. Worked Example — Testing a Business Rule

```
test('BR-06: chunk boundary fires at exactly 10 minutes', () async {
  final fakeClock = FakeClock(start: DateTime(2026, 1, 1));
  final useCase = FinalizeChunkUseCase(repo: FakeRecordingRepository(), clock:
fakeClock);

  fakeClock.advance(Duration(minutes: 10));
  final result = await useCase.checkBoundary();

  expect(result, isA<ChunkBoundaryReached>());
});
```

A fake, injectable clock (never `DateTime.now()` called directly inside a use-case) is what makes a 10-minute business rule testable in milliseconds rather than requiring an actual 10-minute test run — this pattern is itself a Chapter 3.7 coding-standard requirement, not just a testing convenience.

3. Testing State (Riverpod)

```
test('checklist fails with BatteryTooLow when battery < 20%', () async {
  final container = ProviderContainer(overrides: [
    batteryServiceProvider.overrideWithValue(FakeBatteryService(percent: 14)),
  ]);
  final result = await container.read(checklistNotifierProvider.notifier).run();
  expect(result, isA<AsyncError>());
});
```

4. What Unit Tests Deliberately Don't Cover

- Real camera behavior, real background upload, real platform-channel calls — these are faked/mockd at the Platform Services boundary (Volume 3, Chapter 3.4), never exercised for real in a unit test. Real-device verification is Chapter 9.9's job.
- Visual/pixel correctness — that's golden testing, Chapter 9.7.

CHAPTER 9.7

Integration Testing

End-to-End Flows and Design System Visual Regression

Document Title	Integration Testing
Chapter Reference	9.7
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Key End-to-End Flows (integration_test)

Flow	Steps	Traces To
Collector records and uploads a session	Login → Checklist passes → Record → auto chunk boundary → Stop → Session Complete	Volume 1, Ch.1.8, UC-01/02
Offline recording, resumed upload	Airplane mode on → record → chunks Queued → airplane mode off → chunks auto-resume	FR-UPL-05, NFR-AVL-02
Checklist failure and recovery	Battery mocked low → Checklist fails with named cause → condition resolved → re-run passes	FR-CHK-05, Chapter 2.9
Admin assigns a Collector	Create Project → Create Task → Assign Collector → Collector sees it appear	FR-ADM-01-03, Ch.1.8 UC-07
Failed upload, manual retry	Force a 5xx → chunk shows Failed → tap Retry → succeeds	FR-UPL-07, Volume 5 Ch.5.13

Each flow runs against a fake backend (a local mock server returning scripted responses) rather than the real staging environment — fast, deterministic, and runnable in CI (Volume 7, Chapter 7.13) without network flakiness.

2. Golden Tests (Visual Regression)

- Every reusable Design System component (Volume 2, Chapter 2.8 — buttons, status pills, checklist items, form fields) has a golden test capturing its rendered output in both light and dark theme.
- A golden test failing on an unintentional pixel diff is a hard CI gate — intentional visual changes require explicitly regenerating and committing the new golden image as part of that PR, so no visual regression ships silently.

3. What Integration Tests Deliberately Don't Cover

Real camera/lens hardware, real background upload across an actual app kill, real multi-day battery/thermal behavior: these can't be faked meaningfully — they're Chapter 9.8 (Manual Testing) and Chapter 9.9 (Device Testing)'s responsibility instead.

VOLUME 9 — QUALITY ASSURANCE

CHAPTER 9.8

Manual Testing

What Only a Human, on Real Hardware, Can Actually Verify

Document Title	Manual Testing
Chapter Reference	9.8
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Why Manual Testing Still Exists Given Chapters 9.6/9.7

Volume 7, Chapter 7.4 already flagged it: the emulator cannot exercise a real ultra-wide lens, and no automated test can meaningfully assert 'the video actually looks right' or 'the phone didn't get uncomfortably hot after 40 minutes of back-to-back recording.' This chapter is the test plan for exactly those cases.

2. Manual Test Plan

Scenario	What to Check	Why Automation Can't
Real wide-angle capture, multiple device models	Correct 0.5x/0.6x field of view per Volume 5, Chapter 5.1's capability ladder tiers	No emulator has a real ultra-wide sensor (Volume 7, Ch.7.4).
Background upload survives app kill	Force-close the app mid-upload; relaunch; chunk resumes or is safely Queued (never lost or duplicated)	Requires the real Android foreground service / iOS background URLSession (Volume 5, Ch.5.11), not a test-harness fake.
Sustained recording, thermal/battery	Multiple consecutive 10-minute chunks; watch for thermal throttling or unexpected shutdown	Real hardware heat/battery behavior isn't meaningfully simulatable.
Real offline field conditions	Airplane mode in an actual low-signal area, not just a simulator toggle	Real-world radio behavior differs from simulator connectivity toggles.
Accessibility pass	Volume 2, Chapter 2.10's checklist — screen reader, touch targets, contrast in direct sunlight	Requires a human perceiving the actual rendered UI on real hardware.

3. User Acceptance Testing (UAT)

Who signs off: the project owner (or a designated Admin-role tester) runs the Collector and Admin flows end-to-end on real devices before any release — mirroring Volume 0's own "never proceed without explicit approval" principle, now applied to shipping instead of documentation.

What UAT confirms that automated tests can't: that the product actually feels right in the hand — the same qualitative bar Volume 2's UX guidelines (Chapter 2.9) were written against, now checked against the real, built thing rather than a wireframe.

4. What This Chapter Defers

- The specific device models this plan runs against — Chapter 9.9 (Device Testing).

Device Testing

The Device/OS Matrix Every Release Is Verified Against

Document Title	Device Testing
Chapter Reference	9.9
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Minimum Supported OS Versions

Android 10+ (API 29+): chosen because it's the first version with File-Based Encryption as the default (Volume 8, Chapter 8.2's ADR-011 relies on this), and covers the large majority of active Android devices.

iOS 15+: a conservative, still-current baseline; Volume 0's iOS-secondary priority means this matrix is deliberately less exhaustive on the iOS side than Android's.

2. Android Device Matrix (Primary Focus)

Tier	Example Device	Why Included
Reference mid-range	Pixel 6a-class device	Matches Chapter 9.4's performance targets' baseline device.
Low-end	A budget device at the minimum supported OS version	Verifies Chapter 9.4's memory/thermal targets hold at the low end, not just on a comfortable reference device.
Manufacturer variant #1 (e.g. Samsung)	A recent Galaxy A-series or similar	Manufacturer camera-API customizations are exactly where Volume 5, Chapter 5.1's lens-selection capability ladder is most likely to hit its Tier 2/3 fallback — must be verified, not assumed.
Manufacturer variant #2 (e.g. Xiaomi/OnePlus)	A recent mid-range device	Same rationale — camera hardware/firmware fragmentation is the single highest-risk area flagged since Volume 3.

3. iOS Device Matrix (Secondary)

- One current-generation iPhone and one older-but-still-supported model (matching the iOS 15+ floor) — run via the cloud Mac CI service (Volume 7, Chapter 7.8), sufficient given Volume 0's stated secondary priority for iOS during MVP.

4. Automated Device Coverage

Firebase Test Lab (already in the Firebase stack, Volume 4 Chapter 4.1) runs the Chapter 9.7 `integration_test` suite across a wider automated device/OS matrix on every release candidate, catching gross compatibility failures — this supplements, but does not replace, the manual matrix in Sections 2–3, since Test Lab still can't validate real camera output quality any better than the emulator can (Volume 7, Chapter 7.4).

5. Release Gate

A release (Volume 10) does not proceed to store submission until every device in Sections 2–3 has passed Chapter 9.8's manual test plan on the current release candidate build — this is the concrete, checkable gate Chapter 9.5's testing pyramid ultimately exists to support.